Exp No: 12	End-End Communication at Transport Layer - Echo Client Server and Chat

AIM:

To implement (a) an echo client-server using TCP and UDP sockets and (b) a chat program using socket programming, for end-to-end communication at the transport layer.

ALGORITHM / PROCEDURE:

(a1) TCP echo server and client

1. **Server:** create a TCP socket with `socket(AF_INET, SOCK_STREAM)`, `bind()` it to 127.0.0.1 and port 5000, call `listen()` and wait in `accept()` for the client.
2. **Client:** create a TCP socket and call `connect()` with the IP address and port of the server.
3. The client reads a message from the user and sends it with `send()`. The server receives it with `recv()`, displays it and sends the same data back.
4. The client displays the echoed message. The process is repeated until the user types `exit`, then the sockets are closed.

(a2) UDP echo server and client

1. **Server:** create a UDP socket with `SOCK_DGRAM` and `bind()` it to port 5001. No `listen()` / `accept()` is needed because UDP is connectionless.
2. **Client:** create a UDP socket and send the message to the server address with `sendto()`.
3. The server receives the datagram and the client address with `recvfrom()` and sends it back using `sendto()`.
4. The client receives the echo with `recvfrom()` and displays it. The word `exit` closes the sockets.

(b) Chat program

1. The chat server creates a TCP socket on port 5002 and waits for a client. The client connects to it.
2. The client types a message and sends it. The server displays it, then the server user types the reply and sends it back.
3. Messages are exchanged alternately until either side sends the word `exit`. Then both sockets are closed.

Point	TCP	UDP
Socket type	SOCK_STREAM	SOCK_DGRAM
Connection	Connection oriented (3-way handshake)	Connectionless
Reliability	Reliable, ordered delivery	No guarantee
Server calls	bind, listen, accept, recv, send	bind, recvfrom, sendto
Used in	HTTP, FTP, SMTP	DNS, DHCP, streaming

DIAGRAM:

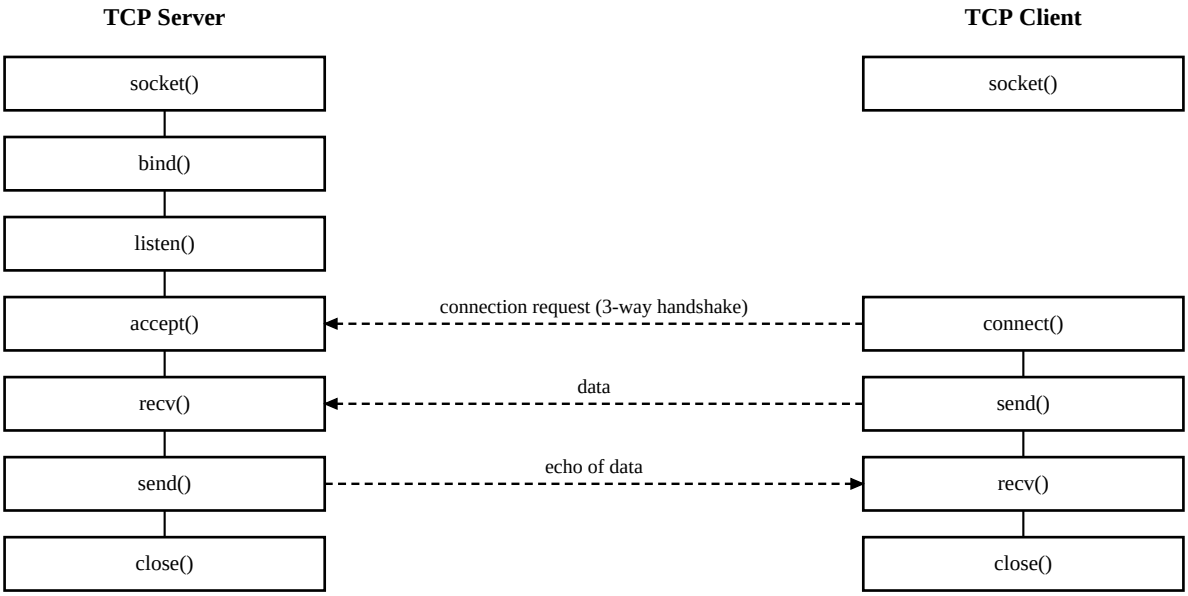

Fig 1: Socket calls in TCP client-server communication

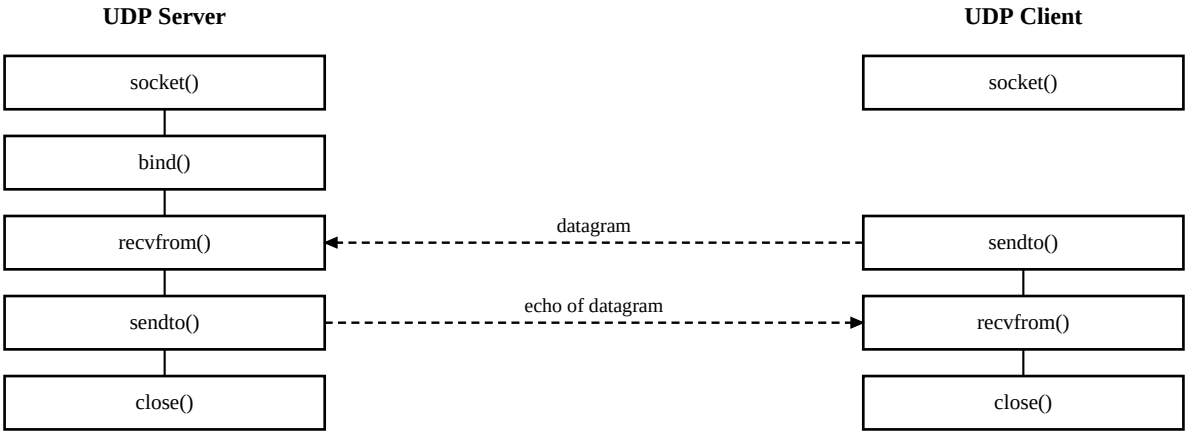

Fig 2: Socket calls in UDP client-server communication

PROGRAM:

tcp_server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5000))
server.listen(1)
print("TCP Echo Server started...")
conn, addr = server.accept()
print("Connected:", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Client:", data.decode())
    conn.send(data)
conn.close()
server.close()
```

tcp_client.py

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
print("Connected to TCP Echo Server")
while True:
    msg = input("Enter message: ")
    if msg.lower() == "exit":
        break
    client.send(msg.encode())
    data = client.recv(1024)
    print("Server:", data.decode())
client.close()
```

udp_server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("127.0.0.1", 5001))
print("UDP Echo Server started...")
while True:
    data, addr = server.recvfrom(1024)
    print("Client:", data.decode())
    if data.decode().lower() == "exit":
        break
    server.sendto(data, addr)
server.close()
```

udp_client.py

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
print("Connected to UDP Echo Server")
while True:
    msg = input("Enter message: ")
    client.sendto(msg.encode(), ("127.0.0.1", 5001))
    if msg.lower() == "exit":
        break
    data, addr = client.recvfrom(1024)
    print("Server:", data.decode())
client.close()
```

chat_server.py

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("127.0.0.1", 5002))
server.listen(1)
print("Chat Server started...")
conn, addr = server.accept()
print("Connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    if msg.lower() == "exit":
        break
    print("Client:", msg)
    reply = input("You: ")
    conn.send(reply.encode())
    if reply.lower() == "exit":
        break
conn.close()
server.close()
```

chat_client.py

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5002))
print("Connected to Chat Server")
while True:
    msg = input("You: ")
    client.send(msg.encode())
    if msg.lower() == "exit":
        break
    reply = client.recv(1024).decode()
    print("Server:", reply)
    if reply.lower() == "exit":
        break
client.close()
```

OUTPUT:

(a1) TCP echo

Terminal 1 (Linux) - python3 tcp_server.py

```
TCP Echo Server started...
Connected: ('127.0.0.1', 33240)
Client: Hello Server
Client: Computer Networks Lab
```

Terminal 2 (Linux) - python3 tcp_client.py

```
Connected to TCP Echo Server
Enter message: Hello Server
Server: Hello Server
Enter message: Computer Networks Lab
Server: Computer Networks Lab
Enter message: exit
```

(a2) UDP echo

Terminal 1 (Linux) - python3 udp_server.py

```
UDP Echo Server started...
Client: Hello UDP
Client: Datagram test
Client: exit
```

Terminal 2 (Linux) - python3 udp_client.py

```
Connected to UDP Echo Server
Enter message: Hello UDP
Server: Hello UDP
Enter message: Datagram test
Server: Datagram test
Enter message: exit
```

(b) Chat program

Terminal 1 (Linux) - python3 chat_server.py

```
Chat Server started...
Connected: ('127.0.0.1', 41436)
Client: Hi Server, how are you?
You: Hello, I am fine
Client: Is the CN record ready?
You: Yes, submit it on Friday
```

Terminal 2 (Linux) - python3 chat_client.py

```
Connected to Chat Server
You: Hi Server, how are you?
Server: Hello, I am fine
You: Is the CN record ready?
Server: Yes, submit it on Friday
You: exit
```

RESULT:

Thus the echo client-server programs using TCP and UDP sockets and the chat program using socket programming were implemented.